

VIP C2 PROGRAMME — FULL STACK DEVELOPMENT WITH MERN

Full Stack Development (FSD) Documentation

ShopEZ — MERN Stack E-commerce Application

Phase: Project Documentation

Document Version: 1.1

Date: 18th of june, 2026

Prepared by: V S S S Manikanta

Classification: Project Documentation — For Academic Submission

Full Stack Development with MERN — Project Documentation

1. Introduction

- **Project Title**

ShopEZ — A Production-Grade Full-Stack MERN E-commerce Platform

- **Team Members**

- **V S S S Manikanta** — Lead Full-Stack Developer & Solutions Architect
 - *Role:* End-to-end implementation of backend API, database modeling, frontend SPA development, security orchestration, state management, and multi-service integration.

2. Project Overview

- **Purpose**

ShopEZ is developed to address key operational challenges in the modern e-commerce landscape. For small and medium-sized merchants, it provides a highly customizable and scalable online storefront to showcase products, manage inventory, and track orders. For end-consumers, it offers a fast, intuitive, and secure shopping experience with dynamic catalog filtering, persistent state retention, and multi-gateway payment processing.

• Features

- **User Management & Authentication:** JWT sessions stored in secure HTTP-only cookies, combined with BCrypt password hashing and role-based access checks (Admin vs. Customer).
- **Dynamic Product Catalog:** Pagination, keyword search, and filters (by category, price range, and rating) working without full-page reloads.
- **Persistent Shopping Cart:** Syncs shopping cart items directly with MongoDB to retain user selections across multiple devices.
- **Secure Checkout Flow:** A multi-step checkout wizard integrated with Stripe/Razorpay for secure card and UPI payment processing.
- **Admin Management Panel:** Full CRUD operations for products, sales metrics charts, and status tracking for order fulfillment.
- **Media Handling:** Ephemeral uploads optimized and served via the Cloudinary CDN.

3. Architecture

• Frontend

The client is structured as a single-page application (SPA) using **React.js**. It features:

- **Context API:** Handles global state for user authentication sessions and shopping cart items.
- **Axios Interceptors:** Manages REST API communication, injecting credentials automatically and centralizing error notifications.
- **Responsive Styling:** CSS variables and HSL palettes optimized for mobile, tablet, and desktop viewports.

• Backend

The server runtime is powered by **Node.js** and structured using **Express.js** in a strict Model-View-Controller (MVC) architecture:

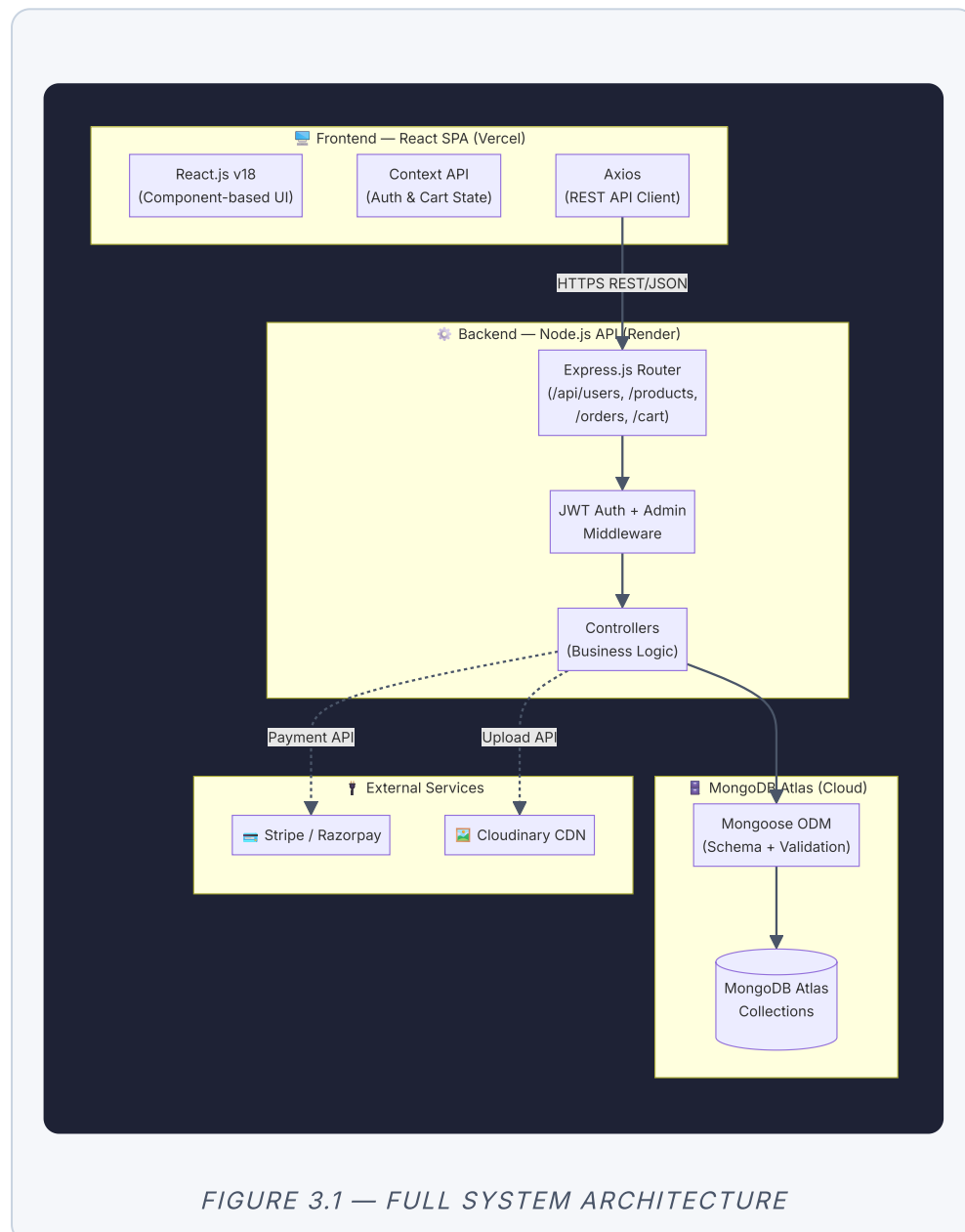
- **Routing:** Decoupled endpoints for users, products, and orders.

- **Middleware:** Intercepts requests for authentication (`protect`), role checking (`isAdmin`), and global exception handling (`errorMiddleware`).

• Database

We utilize **MongoDB Atlas** as our cloud NoSQL data store, modeled and queried using the **Mongoose ODM**:

- **Data Integrity:** Enforces strict schemas and validates inputs before writing to the collections.
- **Referential Constraints:** Uses Mongoose ObjectId references to connect users, carts, products, reviews, and orders.



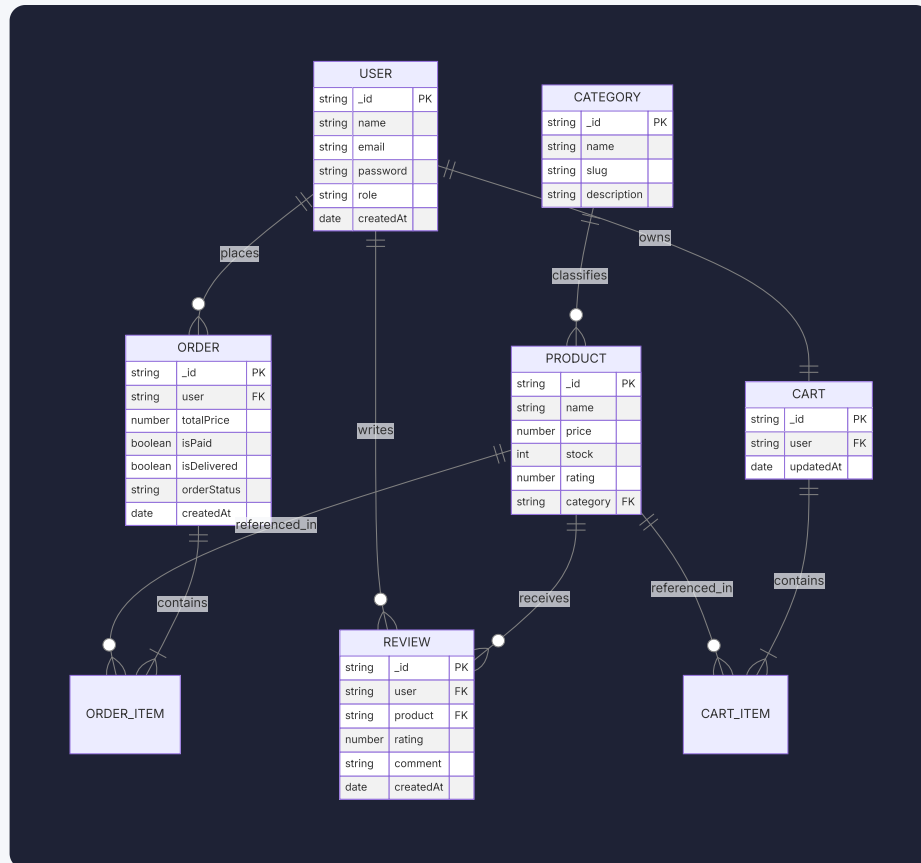


FIGURE 3.2 — ENTITY RELATIONSHIP DIAGRAM



FIGURE 3.3 — UML CLASS DIAGRAM: CORE MONGOOSE MODELS

4. Setup Instructions

• Prerequisites

- **Node.js:** v18 LTS or above installed.
- **MongoDB:** A cloud cluster on MongoDB Atlas or local installation (v6.x+).
- **Package Manager:** npm v9.x or above.
- **External accounts:** Stripe account (for API test keys) and Cloudinary account (for media uploads).

• Installation

1. Configure Environment Variables:

Create a `.env` file under the `/server` folder with the following variables:

```
PORT=8000
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mo
JWT_SECRET=your_jwt_secret_key
STRIPE_SECRET_KEY=sk_test_your_stripe_key
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_api_secret
```

2. Install Dependencies & Seed Database:

```
# Install server dependencies & seed DB
cd server && npm install && npm run seed

# Install client dependencies
cd ../client && npm install
```

5. Folder Structure

- Client

```
client/  
├─ public/           # Static assets  
└─ src/  
    ├─ assets/       # Global CSS and themes  
    ├─ components/   # Reusable presentation parts (Nav  
    ├─ context/       # AuthContext.jsx & CartContext.js  
    ├─ pages/         # View routes (Home, ProductDetail  
    ├─ App.jsx        # React routing setup  
    └─ main.jsx       # DOM mounting entry point
```

- Server

```
server/  
├─ config/           # db.js (Mongoose connection pool  
├─ controllers/      # User, product, and order control  
├─ middleware/       # authMiddleware.js & errorMiddlew  
├─ models/           # Mongoose schema definitions (Use  
├─ routes/           # Express REST endpoint routes  
├─ seeder.js         # Database seeding script  
└─ server.js         # Application entry file
```

6. Running the Application

To run the application locally in development mode, start both the backend API server and frontend development server:

- **Backend Server:** Run in the `/server` directory:

```
npm start
```

(Default port: `http://localhost:8000`)

- **Frontend Server:** Run in the `/client` directory:

```
npm start
```

(Default port: <http://localhost:5173>)

7. API Documentation

Method	Endpoint	Auth Required	Description
POST	/api/users/register	No	Register a new user account
POST	/api/users/login	No	Authenticate user & issue JWT cookie
POST	/api/users/logout	Yes	Invalidate user session & clear cookie
GET	/api/users/profile	Yes (Customer)	Retrieve authenticated user profile
PUT	/api/users/profile	Yes (Customer)	Update user profile data
GET	/api/users	Yes (Admin)	Retrieve a list of all users
DELETE	/api/users/:id	Yes (Admin)	Remove a user account
GET	/api/products	No	Get products (supports search, categories, filters)
GET	/api/products/:id	No	Get detailed view of a single product

Method	Endpoint	Auth Required	Description
POST	/api/products	Yes (Admin)	Add a new product listing
PUT	/api/products/:id	Yes (Admin)	Modify a product listing
DELETE	/api/products/:id	Yes (Admin)	Remove a product listing
POST	/api/products/:id/reviews	Yes (Customer)	Add rating and review comments
POST	/api/orders	Yes (Customer)	Place a new order
GET	/api/orders/myorders	Yes (Customer)	Retrieve personal order history
GET	/api/orders/:id	Yes (Customer/Admin)	Retrieve order details by ID
PUT	/api/orders/:id/pay	Yes (Customer)	Process Stripe payment and set paid status
PUT	/api/orders/:id/deliver	Yes (Admin)	Mark order status as delivered
GET	/api/orders	Yes (Admin)	View all customer orders

8. Authentication

Authentication on the ShopEZ platform is stateless and utilizes **JSON Web Tokens (JWT)**.

- **Token Issuance:** When a user registers or logs in, the server generates a token signed with the `JWT_SECRET`.
- **Cookie Storage:** The token is written directly to the response header as an **HTTP-only, SameSite=Strict** cookie. This prevents cross-site scripting (XSS) extraction and cross-site request forgery (CSRF) vulnerabilities.
- **Authorization:** Protected routes use the JWT validation middleware:

```
const protect = async (req, res, next) => {  
  const token = req.cookies.jwt;  
  if (!token) return res.status(401).json({ message: 'Not authorized' });  
  const decoded = jwt.verify(token, process.env.JWT_SECRET);  
  req.user = await User.findById(decoded.userId).select('-password');  
  next();  
};
```

9. User Interface

The following screenshots showcase different UI features of the ShopEZ application:

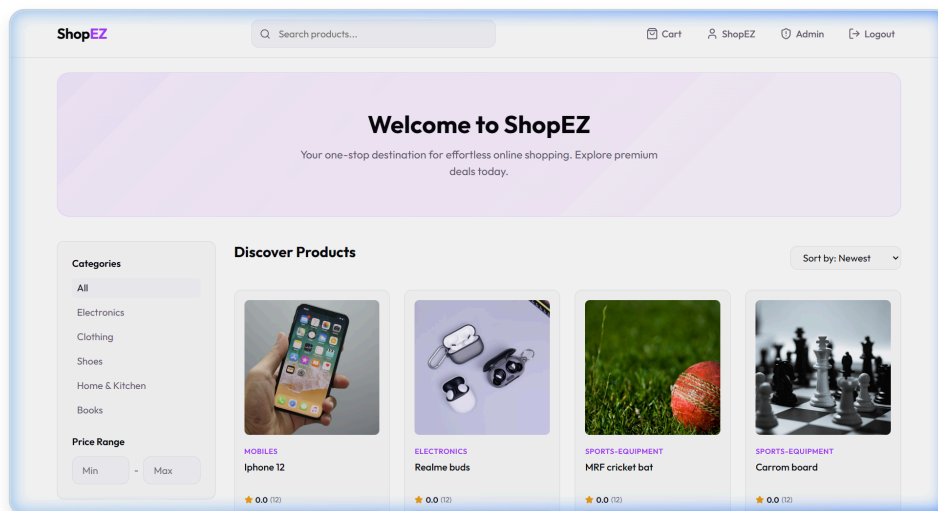


FIGURE 9.1 — SHOPEZ HOMEPAGE AND PRODUCT DISCOVERY GRID

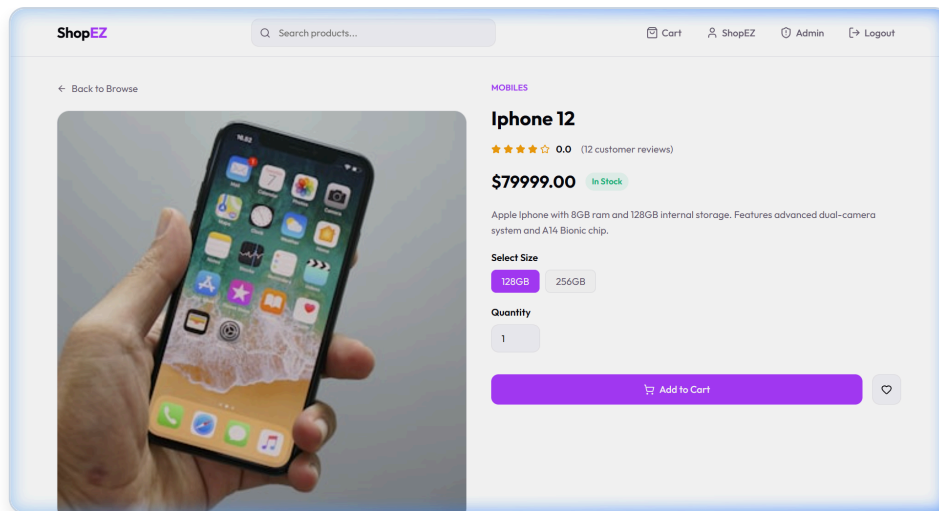


FIGURE 9.2 — PRODUCT DETAIL PAGE WITH RATING & REVIEW FORM

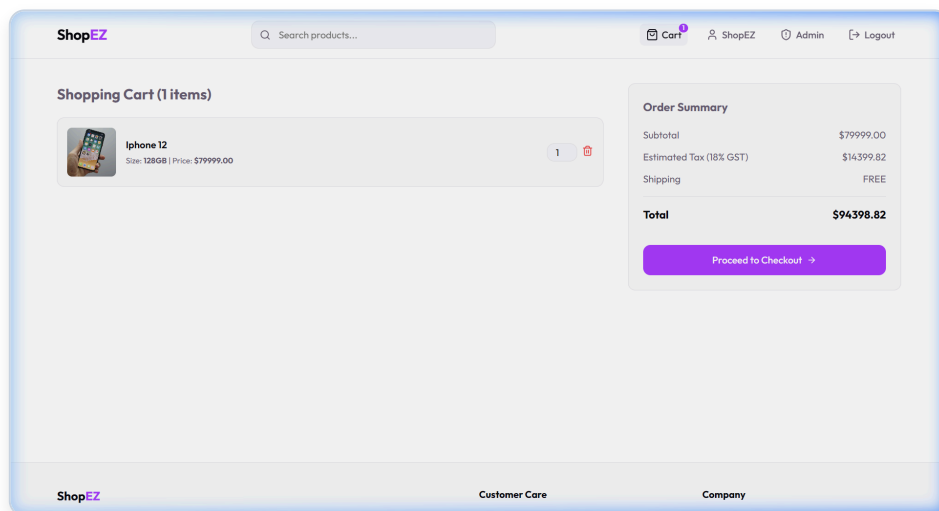


FIGURE 9.3 — SHOPPING CART SUMMARY AND PRICE ESTIMATION

10. Testing

The testing strategy for ShopEZ ensures reliability and correctness across multiple levels:

- **Unit Testing:** Testing of utilities, validation rules, and context status reducers (Jest, RTL).
- **Integration Testing:** Running API request/response assertions using Postman and Newman collections to verify route security and status codes.
- **User Acceptance Testing (UAT):** Execution of 16 defined manual UAT scripts covering complete user flow (Registration → Search → Cart → Checkout → Payment → Admin Inventory verification).

11. Screenshots or Demo

The platform is deployed online in a decoupled cloud environment. You can access the application and review the administrative features using the following credentials:

Resource	URL / Details
Live Storefront	<code>https://e-commerce-application-neon-five.vercel.app/</code>
Backend API URL	<code>https://shopez-api-c30e.onrender.com</code>
Customer Account	<code>test@email.com</code> / <code>password123</code>
Admin Account	<code>admin@shopez.com</code> / <code>adminpassword123</code>

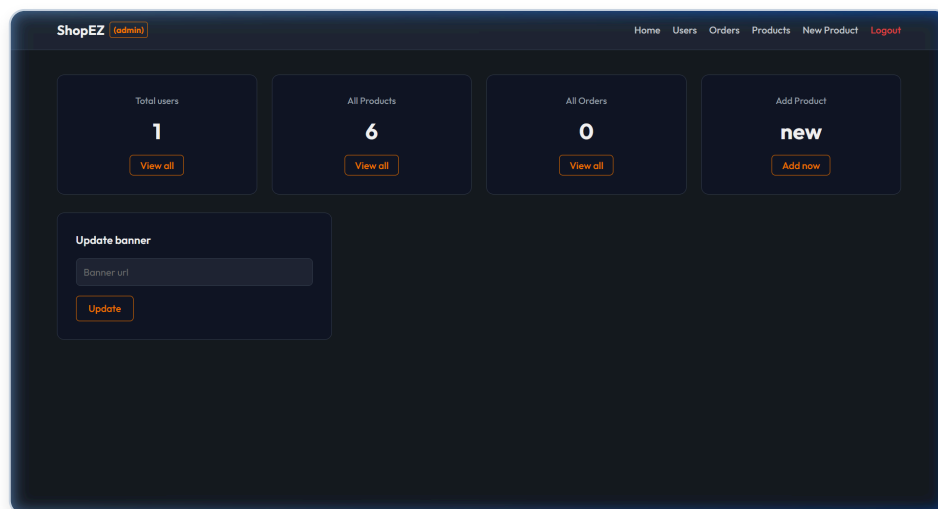
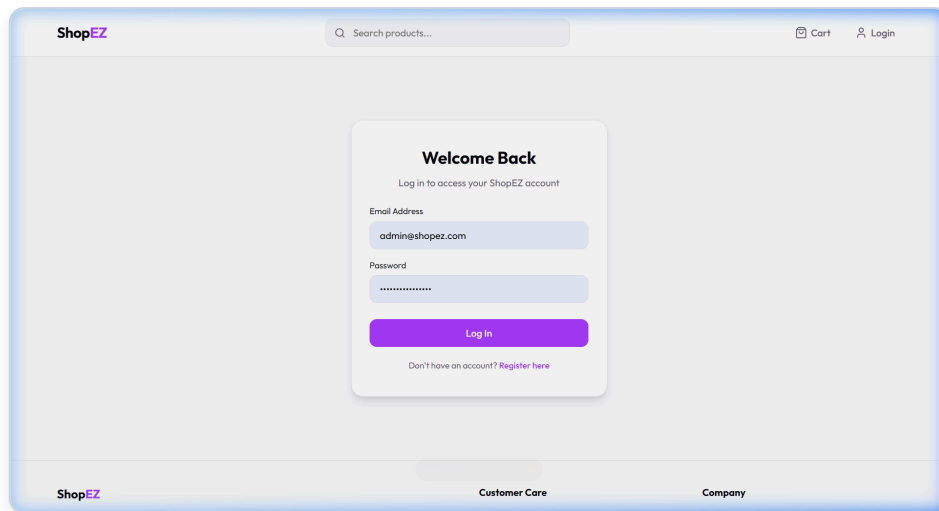


FIGURE 11.1 — ADMIN PORTAL: PRODUCT CRUD AND INVENTORY ANALYTICS



The image shows a web browser window with the ShopEZ logo in the top left corner. A search bar with the placeholder text "Search products..." is located in the top center. In the top right corner, there are links for "Cart" and "Login". The main content area features a "Welcome Back" login form. The form includes a heading "Welcome Back", a subheading "Log in to access your ShopEZ account", and two input fields: "Email Address" with the value "admin@shopez.com" and "Password" with masked characters. A purple "Log In" button is positioned below the password field. At the bottom of the form, there is a link that says "Don't have an account? Register here". The footer of the page contains the ShopEZ logo on the left and links for "Customer Care" and "Company" on the right.

ShopEZ

Search products...

Cart Login

Welcome Back

Log in to access your ShopEZ account

Email Address

admin@shopez.com

Password

.....

Log In

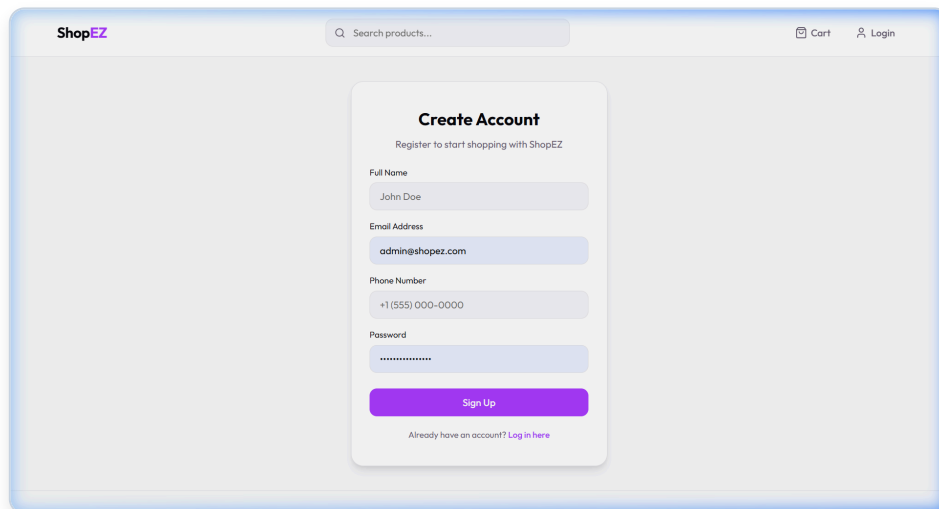
Don't have an account? [Register here](#)

ShopEZ

Customer Care

Company

FIGURE 11.2 — CUSTOMER LOGIN INTERFACE



The image shows a web browser window with the ShopEZ logo in the top left corner. A search bar with the placeholder text "Search products..." is located in the top center. In the top right corner, there are links for "Cart" and "Login". The main content area features a "Create Account" registration form. The form includes a heading "Create Account", a subheading "Register to start shopping with ShopEZ", and four input fields: "Full Name" with the value "John Doe", "Email Address" with the value "admin@shopez.com", "Phone Number" with the value "+1 (555) 000-0000", and "Password" with masked characters. A purple "Sign Up" button is positioned below the password field. At the bottom of the form, there is a link that says "Already have an account? Log in here". The footer of the page contains the ShopEZ logo on the left and links for "Customer Care" and "Company" on the right.

ShopEZ

Search products...

Cart Login

Create Account

Register to start shopping with ShopEZ

Full Name

John Doe

Email Address

admin@shopez.com

Phone Number

+1 (555) 000-0000

Password

.....

Sign Up

Already have an account? [Log in here](#)

ShopEZ

Customer Care

Company

FIGURE 11.3 — CUSTOMER ACCOUNT REGISTRATION INTERFACE

12. Known Issues

Issue ID	Description	Mitigation / Plan
KI-01	Cart state may desync on concurrent multi-device sessions.	Server-side updatedAt timestamp comparison on cart save.
KI-02	Render free-tier cold-start causes 30–60s initial delay.	Upgrade to paid tier for production hosting.
KI-03	No email verification on user registration.	Add Nodemailer-based verification flow in future release.

13. Future Enhancements

Priority	Enhancement	Description
Medium	Email Alerts	Nodemailer order dispatch confirmations.
Low	Wishlists	Saved items library for customer accounts.
Future	AI Recommendations	Item-based collaborative filtering suggestions.
Future	React Native Companion	Mobile application utilizing existing REST API.